

Take-home Assignment: Prediction Market Aggregator

Context

Prediction markets allow users to buy and sell outcome shares on real-world events. Prices change in real time as new orders arrive.

Your task is to build a **simple web application** that helps a user understand **liquidity and pricing** for a single prediction market by combining data from **two different venues**.

You may use **any framework, libraries, or tools** you feel are appropriate.

Relevant background reading:

- Polymarket overview:
<https://docs.polymarket.com/polymarket-learn/get-started/what-is-polymarket>
 - Kalshi WebSocket order book updates:
<https://docs.kalshi.com/websockets/orderbook-updates>
-

What the app should do

1. Market view

The app should display a single prediction market with two outcomes (e.g. *“Will JD Vance win the 2028 US Presidential Election?”*).

For the selected market:

- Show the current **order book** for one outcome.
- Prices and sizes should update **in real time** as new data arrives.
- The order book should reflect **combined liquidity** from both venues.

Users should be able to visually understand:

- where liquidity is concentrated

- how prices differ across venues
 - how the combined book compares to either venue individually
-

2. Live aggregation

The data backing the order book should come from **both Polymarket and Kalshi**.

The app should:

- listen for live updates from each venue
- reconcile differences in how each venue represents order book data
- present a **single coherent view** to the user

If one venue temporarily stops sending updates, the app should continue to function and make this visible to the user.

3. Quote experience

The app should allow a user to:

- enter a dollar amount they want to spend
- select which outcome they want exposure to

The app should then:

- calculate how many shares the user would receive at current prices
- assume the order may be filled across **both venues**
- clearly show how the fill is split between venues

This is a **pricing / quoting exercise only** — no real orders need to be placed.

4. Long-running behavior

Assume this app may be left open for long periods of time.

It should:

- continue updating correctly over time
 - not degrade in responsiveness or correctness
 - behave sensibly if connections are interrupted and later resume
-

Constraints & expectations

- The focus is on **frontend correctness, clarity, and robustness**
 - You may implement a minimal backend if you believe it's necessary
 - You are free to mock or simplify anything that is not essential to the core experience
 - You are encouraged to use AI tools to move faster, but the final result should reflect your own judgment
-

Deliverables

1. A GitHub repository with:
 - the working application
 - clear setup instructions
 2. A short README describing:
 - key design decisions you made
 - assumptions and tradeoffs
 - what you would improve or extend with more time
-

Time guidance

We expect this to take **a few focused hours**. Depth and thoughtfulness matter more than completeness.